

Train Vehicle Number Recognition and Consist Counting Using Deep Learning

SS.Ntuli

Transnet, Johannesburg, South Africa

ABSTRACT: In this work, an efficient method for counting and identifying train vehicle numbers using computer vision and deep learning is proposed. The method serves as an alternative or backup for the widely used radio frequency identification (RFID) based train vehicle identification systems. The method consists of 2 deep learning models and a combination of text detection and recognition. The two deep learning models leverages the VGG-16 pretrained convolutional layer as the base for their fine-tuned fully connected layers. The 1st model classifies if there is a gap between train vehicles or when there is only a vehicle in the region of interest. A transition from gap to vehicle class increments the train vehicle counts in the video stream. The 2nd model accepts the region of interest that is classified as a vehicle to distinguish if the frame consists of a locomotive or a wagon. Text regions in a frame classified as vehicle, are detected by the efficient and accurate scene text (EAST) detector. Tesseract OCR (optical character recognition) extracts text information, that satisfies the set (wagon or locomotive) number format, from the detected regions. The Gap/Vehicle classifier achieved an accuracy of 99.70% whilst the vehicle type classifier achieved 99.65%.

1 INTRODUCTION

RFID systems are widely used in the railway networks for train vehicle identification. All vehicles within a train are fitted with RFID tags which are read by the vehicle identification system readers installed trackside, in strategic locations. The tag hosts information pertaining to the vehicle (i.e vehicle type, number and the owner). The track side RFID readers acquire the information from each vehicle tag. The information is used for tracking the vehicle location as well as to integrate with train condition assessment systems and align the measurements and the generated alarms with the appropriate vehicle.

The challenge that has called for an alternative or secondary vehicle identification method; are the RFID tags which are falling off the vehicles. The challenge has resulted into inefficient train vehicle tracking, vehicle condition analysis and has prolonged the mean time to repair fault that are alarmed by the condition assessment systems - due to the alarm being aligned with a placeholder *w* (for wagons) and *l* (for locomotives) instead of the vehicle number, see position 3 and 13 on figure1.

In the proposed solution, a video camera is used to capture the frames on the side of the vehicle where the stenciling is located (Fig 2). The information stenciled on the vehicle entails the same information as

the one programed into the tag (i.e vehicle type and vehicle number). The method proposed solely focuses

```
2025/04/24 12:11:14
Condition
CTC: kim
Measurement point: SIH542
Optic: 70905
Start time: 2025/04/24 12:05:45
End time: 2025/04/24 12:10:42
Train number: 20250424-120545U
Direction: U
Line: N
Vehicle count: 128
Axle count: 518
Minimum speed (km/h): 10
Maximum speed (km/h): 28.8
Error byte: 137
Errors: Axle Error, Unidentified Vehicles, Tags Missed, No train number from DB
Train number from database: Y
TCO IP address: 0.0.0.0
Vehicles: Position, Start axle, Class, Speed (km/h), Number, Orientation, Tag type, Left reader tag, Reads,
1, 1, z, 10.0, E23111, R, 23E, , 0, 23E 00 E23111 1, 54, 5.949, 1.949, 1.912, 10.079, 1.940, 1.917, 00
2, 7, z, 10.2, E23075, F, 23E, , 0, 23E 00 E23075 2, 49, 5.999, 1.904, 1.897, 10.098, 1.901, 1.914, 00
3, 13, z, 10.4, l, , , 0, , 0, 4.057, 1.932, 1.892, 10.145, 1.943, 1.888, —
4, 19, z, 10.5, 60144378, R, CR10, , 0, CR10 00 60144378 1, 67, 2.144, 1.806, 4.478, 1.799, 00
5, 23, z, 10.6, 60810068, F, CR21, , 0, CR21 00 60810068 2, 68, 2.143, 1.819, 4.501, 1.807, 00
6, 27, z, 10.7, 60271671, R, CR16, , 0, CR16.. 00 60271671 1, 73, 2.128, 1.811, 4.509, 1.813, 00
7, 31, b, 10.8, 60599146, F, CR17, , 0, CR17 00 60599146 2, 74, 2.104, 1.818, 4.533, 1.815, 00
8, 35, z, 10.8, 60143827, R, CR9, , 0, CR9 00 60143827 1, 24, 2.146, 1.800, 4.490, 1.813, 00
9, 39, z, 10.9, 60566345, R, CR17, , 0, CR17 00 60566345 1, 68, 2.094, 1.806, 4.502, 1.808, 00
10, 43, z, 11.0, 60570601, R, CR17, , 0, CR17 00 60570601 1, 53, 2.113, 1.814, 4.511, 1.817, 00
11, 47, z, 11.0, 60272058, F, CR16, , 0, CR16.. 00 60272058 2, 67, 2.115, 1.815, 4.509, 1.813, 00
12, 51, z, 11.0, 60598530, F, CR17, , 0, CR17 00 60598530 2, 73, 2.158, 1.810, 4.491, 1.811, 00
13, 55, b, 11.0, w, ?, , 0, , 0, 2.116, 1.803, 4.499, 1.802, —
14, 59, b, 11.0, 60533161, F, CR17, , 0, CR17 00 60533161 2, 60, 2.151, 1.803, 4.538, 1.825, 00
15, 63, z, 11.0, 60553014, R, CR17, , 0, CR17 00 60553014 1, 96, 2.127, 1.811, 4.487, 1.809, 00
16, 67, z, 11.1, 60538406, F, CR17, , 0, CR17 00 60538406 2, 55, 2.165, 1.808, 4.497, 1.809, 00
17, 71, z, 11.1, 60544791, F, CR17, , 0, CR17 00 60544791 2, 72, 2.142, 1.816, 4.510, 1.800, 00
```

Figure 1: Train information file with missing tags

on extracting the vehicle number from the stenciling (which serves as a unique identifier of the vehicle). The stenciling does not only hold the vehicle number but has additional information which includes the wagon type, tare weight, load and company name. The additional information needs to be filtered out and only the vehicle number should be the output of the method. The difference in vehicle number formatting standard for the locomotive and that of the wagon must be considered. The read vehicle number must be

further relayed into the RFID system processor to compensate for the vehicles with missing tags on the train information file.



Figure 2: Stenciled wagon

The frames per vehicle must be traced throughout the video stream to avoid duplicates and to enable efficient counting of the vehicles. The vehicle count/position must be aligned to its corresponding vehicle number in the data frame.

Computer vision and deep learning are the emerging, cost effective and accurate techniques for object detection and text recognition (Vavilin et al. 2022). These methods are utilized in the solution to classify vehicle type, count number of train vehicles, detect and recognize vehicle numbers (stenciled on the side of the vehicle)

The remainder of the paper is arranged as follows. The proposed solution logic will be covered in section 2. Section 3 will cover the data set used to train the models. Section 4 contains experiment conducted, and section 5 concludes and outlines future works in relation to the proposed solution.

2 PROPOSED SOLUTION LOGIC

Figure 4 illustrates the proposed solution logic. The methodology receives a video stream as an input. A text trigger detector variable (which ensures that the preceding frames of the same vehicle are skipped when the vehicle number has been identified) is set to 1. The variables to store the vehicle number string, vehicle count, and vehicle type list are also set. Each video stream frame is processed individually and copy of the original frame is created. The copy will serve as an input during vehicle number detection and identification stage. The original input frame is cropped into a 64x64 pixel frame (region of interest) as illustrated by figure 3. The region of interest size optimizes the method and is compatible with the VGG-16 model since it a multiple of the image size that was used train the VGG-16 model. The region of interest at which the original frame is cropped, is

placed in the middle of the vehicle. The cropped image is used to classify if there is a gap between vehicles or only a vehicle presence.



Figure 3: Crop the original image into 64x64 pixels

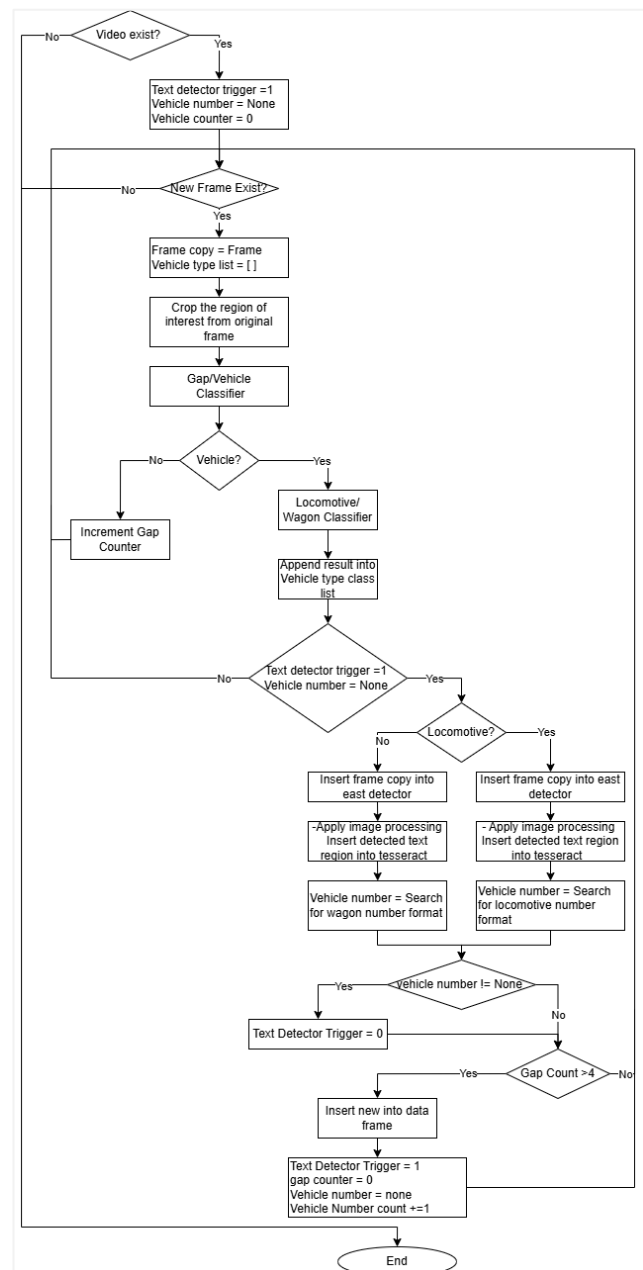


Figure 4: Proposed solution logic

2.1 Gap or Vehicle classification

Transfer learning techniques using pretrained models have emerged as powerful approaches for custom object detection, enabling significant improvements in detection accuracy and efficiency (Mtasher et al. 2024). VGG16 architecture is one of the widely used pre-trained models for image classification (Astuti et al. 2025). Figure 5 shows the standard VGG-16 architecture which mainly consists of two stages, the convolutional layers for feature mapping and a fully connected network for classification. The convolutional

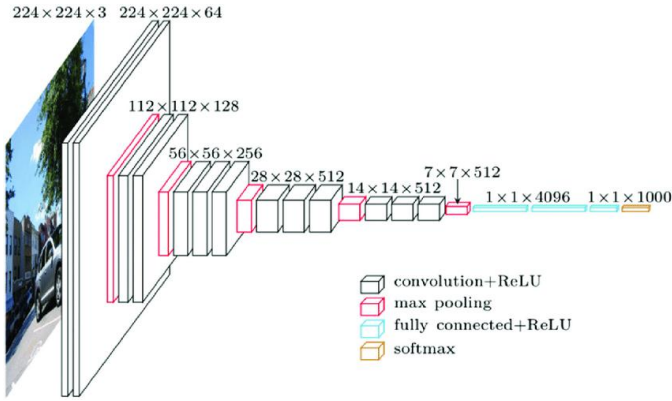


Figure 5: VGG-16 architecture

layer stage consists of a repeating block of two or three convolutional layers followed by a max-pooling layer. The output of the convolutional layer serves as an input to the fully connected network. VGG-16 model was trained on the ImageNet dataset with 1000 output classes therefore it has a low-level knowledge on features like spatial, edges, rotation, lighting, shapes Tammina (2019). The proposed solution leverages the pre-trained VGG-16 model by inheriting its low-level knowledge and fine-tuning the fully connected layer to classify the presence of a train vehicle (Fig 6a) or gap between vehicles (Fig 6b) in the cropped image.



Figure 6a: Vehicle



Figure 6b: Gap

If the classifier classifies the cropped image as a gap, a gap counter is incremented. The gap counter protects the proposed solution from counting the same vehicle twice. When a vehicle is classified, the cropped image is inputted into the locomotive/wagon classifier.

2.2 Locomotive or wagon classifier

Transfer learning technique is also unleashed, by inheriting VGG-16 feature map and fine-tuning the fully connected layer to classify if the cropped vehicle image contains a locomotive (Fig 7a) or a wagon (Fig 7b). The classification results are stored in a vehicle type list for surety of the classification when the vehicle details are inserted into the data frame.



Figure 7a: Locomotive



Figure 7b: Wagon

2.3 Text detection

Zhou et al. (2017) proposed a deep learning pipeline (Fig 8) that yields fast and accurate text detection in a natural scene. The pipeline outperformed state-of-the-art methods in terms of accuracy and efficiency. Hence, the name EAST (efficient and accuracy scene text) detection. The algorithm achieves an F-score of 0.7820 at 13.2fps at 720p resolution, which makes it an ideal option for real time application. The pipeline accepts an image as an input and outputs geometrical points of the regions (where text is detected from an image) and the confidence scores associated with the detected regions.

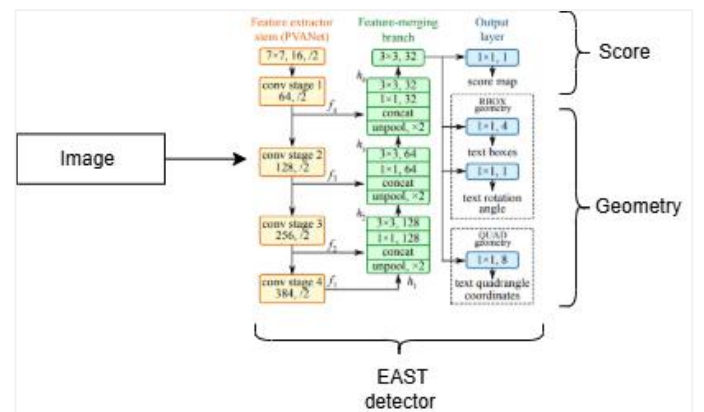


Figure 8: East detector pipeline

The proposed solution incorporates the EAST detector to identify the text regions from the copy of the original video frame, which its cropped image is classified as a vehicle. A minimum confidence of 90% on the non-maximum suppression is set, to eliminate as many false positives detections as possible.



Figure 9: Detected text region

2.4 Tesseract OCR recognition

The detected text regions are cropped out of the original image copy and passed into tesseract (an open-source OCR engine developed by HP Akhil (2016)). Tesseract extract text from the detected regions. Prior to passing the detected regions into the tesseract OCR engine, image processing techniques are applied on the regions to improve the engine's recognition accuracy.

The image processing techniques employed to the regions are as follow:

- Converted the image from RGB to grayscale
- Applied a bilateral filter to denoise the image while preserving the edges
- The image is binarized using adaptive thresholding
- Dilation is applied the on the image to fill broken characters
- The dilated image is then eroded image to remove excess pixel

The resultant pre-processed detected region is as shown by figure 10. Following the pre-processing stage, the region is fed into the tesseract OCR engine.

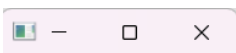


Figure 10: Pre-processed Text region

When a locomotive is detected, the tesseract engine searches for $[1-4][0-9]\backslash s[0-9][0-9][0-9]$ or $[0-9][0-9][0-9][0-9]$ text format and $[0-9][0-9]-[0-9][0-9][0-9]\backslash s[0-9][0-9][0-9]$ for the wagons.

If the tesseract OCR engine finds the vehicle number, the text detector trigger is reset to ensure the current detected vehicle is not reprocessed for system optimization purposes. The gap count is checked if it is greater than 4 to legitimate the new detected vehicle and the details are inserted into the data frame.

The text detector trigger is set to process a new frame, the gap count is reset, vehicle number variable is reset, and vehicle count is incremented.

3 DATASET

3.1 Vehicle and gap classification

The dataset contains a total of 19 522 images of size 64x64 pixels cropped from multiple video streams. 12 384 images are used for training, 3227 for validation and 3911 for testing.

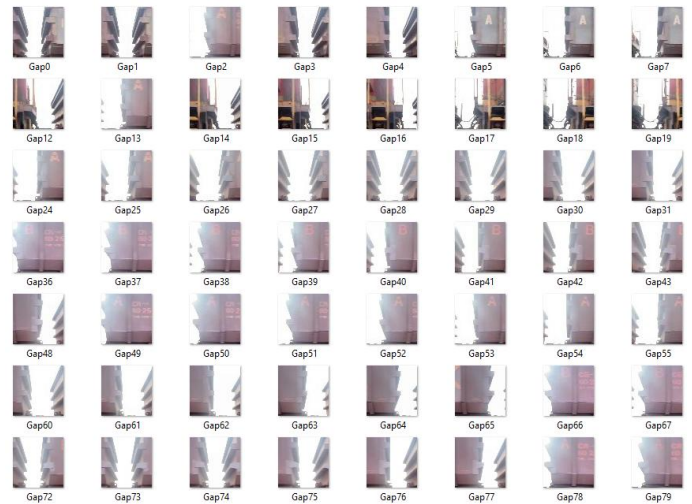


Figure 11: Gap sample dataset

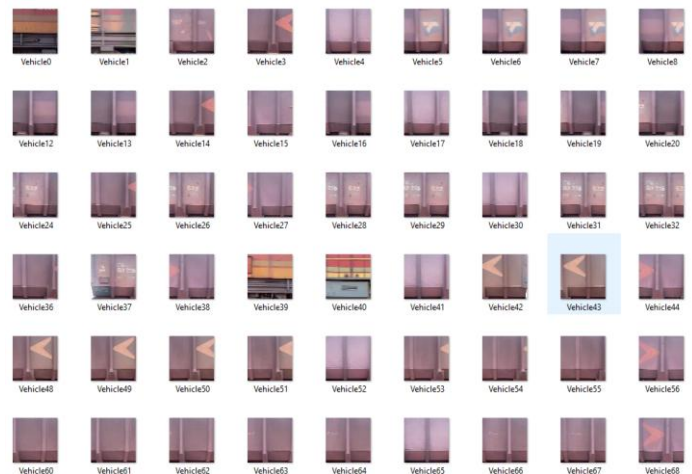


Figure 12: Vehicle sample data set

3.2 Vehicle Type classification

The dataset contains a total of 20340 images of size 64x64 pixels. 10990 images are used for training, 6087 for validation and 3227 for testing.

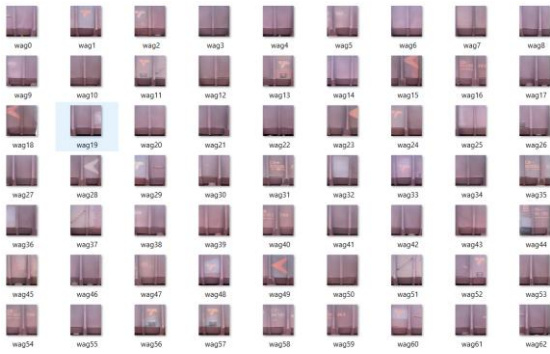


Figure 13: Wagon sample dataset



Figure 14: Locomotive sample dataset

4 EXPERIMENT

4.1 Environment

The experiment was carried out in a 13th Gen Intel(R) Core (TM) i7-13650HX 2.60 GHz 48GB RAM and NVIDIA GeForce RTX 4080

4.2 Model setup

During the training phase, the convolutional layer of the VGG-16 pre-trained model is frozen and only the fully connected network is finetuned to adapt to the vehicle/gap classification. Figure 15 shows the layout of a fine-tuned fully connected network which has as input, the flattened output of the feature map, two Relu activated hidden layers with 20 neurons each and a softmax activated output layer for two categories (gap and vehicle). The network is also fitted with a 20% dropout at each hidden layer to avoid the model from overfitting.

Table 1 shows the selected hyperparameters used to train the fully connected layer and table 2 illustrate the call back parameters to avoid overfitting during training.

```
x = Flatten()(base_model.output)
x = Dense(20,activation='relu')(x)
x = Dropout(0.2)(x)
x = Dense(20,activation='relu')(x)
x = Dropout(0.2)(x)
x = Dense(2,activation='softmax')(x)
```

Figure 15: Fully connected layer for gap/ vehicle classification

Table 1. Training parameters

Parameters	
Optimizer	Adam
Loss function	categorical cross-entropy
Epoch	50

Table 2. Early stop parameters

Parameters	
Monitor	Validation accuracy
Patience	5
Restore weight	True
Mode	max

The same set up is employed on the vehicle type classifier fully connected layer, with minimal variation on the number of neurons on the hidden layers. The 1st hidden layer consists of 10 neurons and a 20% drop out, the 2nd hidden layer contains 20 neurons and 10% dropout. The output layer contains a softmax activation function for the two categories (locomotive or wagon). See figure 16.

```
x = Flatten()(base_model.output)
x = Dense(10,activation='relu')(x)
x = Dropout(0.2)(x)
x = Dense(20,activation='relu')(x)
x = Dropout(0.1)(x)
x = Dense(2,activation='softmax')(x)
```

Figure 16: Fully connected layer for vehicle type classification

The hyperparameter for training and calling back are as illustrated by table 1-2

Table 3. Training parameters

Parameters	
Optimizer	Adam
Loss function	categorical cross-entropy
Epoch	100

Table 4. Early stop parameters

Parameters	
Monitor	Validation accuracy
Patience	5
Restore weight	True
Mode	max

4.3 Results

Due to the configured early stop parameters, the gap or vehicle classifier model stopped training on the 7th epoch and restored the weights of the 3rd epoch which gives an accuracy of 99.70% on the training dataset and 99.97% on the validation dataset (Fig 17-18).

Epoch 1/50	63s 320ms/step - accuracy: 0.9416 - loss: 0.1538 - val_accuracy: 0.9994 - val_loss: 0.0028
104/194	
Epoch 2/50	82s 422ms/step - accuracy: 0.9970 - loss: 0.0116 - val_accuracy: 0.9997 - val_loss: 0.0027
104/194	
Epoch 3/50	189s 972ms/step - accuracy: 0.9978 - loss: 0.0074 - val_accuracy: 0.9994 - val_loss: 0.0029
104/194	
Epoch 4/50	186s 960ms/step - accuracy: 0.9985 - loss: 0.0046 - val_accuracy: 0.9994 - val_loss: 0.0034
104/194	
Epoch 5/50	189s 973ms/step - accuracy: 0.9989 - loss: 0.0037 - val_accuracy: 0.9997 - val_loss: 0.0028
104/194	
Epoch 6/50	72s 368ms/step - accuracy: 0.9992 - loss: 0.0036 - val_accuracy: 0.9997 - val_loss: 0.0031
104/194	
Epoch 7/50	63s 326ms/step - accuracy: 0.9989 - loss: 0.0032 - val_accuracy: 0.9997 - val_loss: 0.0022
104/194	

Figure 17: Training result for gap and vehicle classification

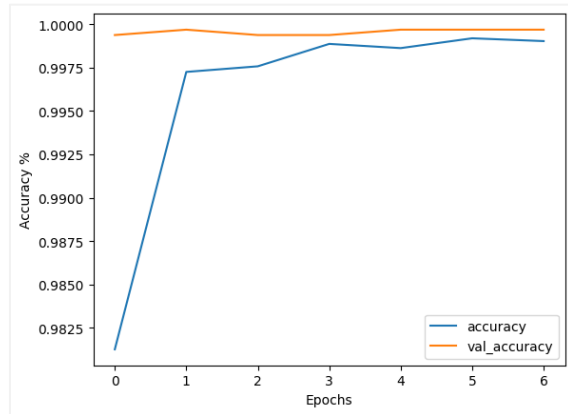


Figure 18: Gap/Vehicle classification per epoch

On the vehicle type classifier model the training is stopped on the 14th epoch and the weights of the 10th epoch are restored. The model achieved an accuracy of 99.65% on the training dataset and 100% accuracy as illustrated by figure 19-20

Epoch 1/100	66s 379ms/step - accuracy: 0.8645 - loss: 0.2985 - val_accuracy: 1.0000 - val_loss: 0.0017
172/172	
Epoch 2/100	70s 405ms/step - accuracy: 0.9876 - loss: 0.0314 - val_accuracy: 1.0000 - val_loss: 3.6082e-04
172/172	
Epoch 3/100	71s 413ms/step - accuracy: 0.9947 - loss: 0.0185 - val_accuracy: 1.0000 - val_loss: 1.8246e-04
172/172	
Epoch 4/100	71s 411ms/step - accuracy: 0.9953 - loss: 0.0147 - val_accuracy: 1.0000 - val_loss: 1.2798e-04
172/172	
Epoch 5/100	69s 403ms/step - accuracy: 0.9944 - loss: 0.0143 - val_accuracy: 1.0000 - val_loss: 2.7067e-04
172/172	
Epoch 6/100	69s 402ms/step - accuracy: 0.9964 - loss: 0.0107 - val_accuracy: 1.0000 - val_loss: 2.1754e-05
172/172	
Epoch 7/100	69s 402ms/step - accuracy: 0.9965 - loss: 0.0097 - val_accuracy: 1.0000 - val_loss: 5.9709e-05
172/172	
Epoch 8/100	69s 400ms/step - accuracy: 0.9947 - loss: 0.0139 - val_accuracy: 1.0000 - val_loss: 1.5976e-05
172/172	
Epoch 9/100	69s 400ms/step - accuracy: 0.9968 - loss: 0.0093 - val_accuracy: 1.0000 - val_loss: 1.5208e-05
172/172	
Epoch 10/100	69s 402ms/step - accuracy: 0.9957 - loss: 0.0121 - val_accuracy: 1.0000 - val_loss: 5.8447e-05
172/172	
Epoch 11/100	69s 402ms/step - accuracy: 0.9954 - loss: 0.0119 - val_accuracy: 1.0000 - val_loss: 6.7422e-05
172/172	
Epoch 12/100	69s 401ms/step - accuracy: 0.9958 - loss: 0.0122 - val_accuracy: 1.0000 - val_loss: 1.2705e-05
172/172	
Epoch 13/100	70s 407ms/step - accuracy: 0.9962 - loss: 0.0101 - val_accuracy: 1.0000 - val_loss: 7.7618e-06
172/172	
Epoch 14/100	68s 404ms/step - accuracy: 0.9959 - loss: 0.0114 - val_accuracy: 1.0000 - val_loss: 2.4351e-05
172/172	

Figure 19: Training result for vehicle type classification

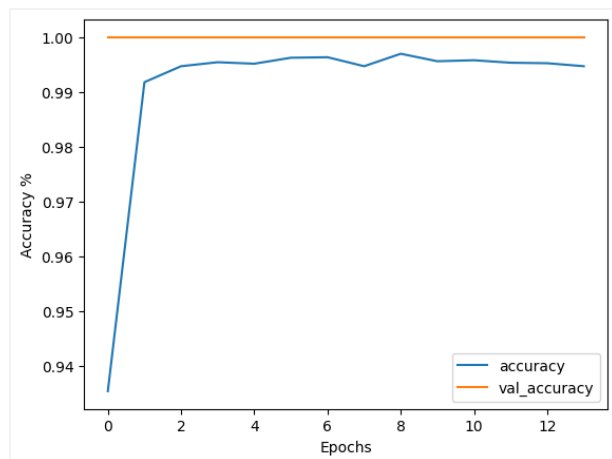


Figure 20: Vehicle type model accuracy per epoch

5 CONCLUSION AND FUTURE WORK

An alternative or secondary train counting and vehicles number recognition using deep learning techniques is set forth. The proposed methodology prioritised system optimization by using transfer learning to train the models, a minimal sized image of 64x64 pixels as inputs into the models and the use of EAST detector for text region detection. The detected regions are further pre-processed through image processing techniques to convert them into binary images. Binarized input images to the tesseract OCR enhances the text recognition accuracy. When the models are trained with the inherited VGG-16 feature map weights, the results for both the gap/vehicle and vehicle type classifiers gives impressive accuracy of 99.70% and 99.97% respectively. As future work, an addition of various wagon and locomotive types would be added into the dataset in varying conditions. Results will be compared against alternative existing pre-trained models to identify a model with the best results and optimization. Integrate the system with the camera trigger to start capturing the video stream when the train present is detected. Integrate the system with the RFID based vehicle identification system to compensate for the missing tags. Migrate the system into an embedded computer board.

6 REFERENCES

- Vavilin, Andrey & Lomov, Andrey & Roman, Titkov. (2022). Real-time Train Wagon Counting and Number Recognition Algorithm. 1-4. 10.1109/IWIS56333.2022.9920835.
- Zhou, Xinyu & Yao, Cong & Wen, He & Wang, Yuzhi & Zhou, Shuchang & He, Weiran & Liang, Jiajun. (2017). EAST: An Efficient and Accurate Scene Text Detector. 10.48550/arXiv.1704.03155.
- Kornprobst, Pierre & Tumblin, Jack & Durand, Frédo. (2009). Bilateral Filtering: Theory and Applications. Foundations and Trends in Computer Graphics and Vision. 4. 1-74. 10.1561/06000000020.
- Mtasher, Ashwaq & Al-wakel, Esraa. (2024). Custom Object Detection Using Transfer Learning with Pretrained Models for Improved Detection Techniques. Journal La Multiapp. 5. 10-18. 10.37899/journallamultiapp.v5i1.843.
- Laroca, Rayson & Boslooper, Alessander & Menotti, David. (2020). Automatic Counting and Identification of Train Wagons Based on Computer Vision and Deep Learning. 10.48550/arXiv.2010.16307.
- Astuti, Erna & Hidayat, Muslim. (2025). Transfer Learning Analysis VGG16 For the Detection of Tuberculosis. Jurnal Teknik Elektro dan Komputer TRIAC. 12. 27-32. 10.21107/triac.v12i1.28672.
- S., Akhil. (2016). Overview of Tesseract OCR engine.
- Tammina, Srikanth. (2019). Transfer learning using VGG-16 with Deep Convolutional Neural Network for Classifying Images. International Journal of Scientific and Research Publications (IJSRP). 9. p9420. 10.29322/IJSRP.9.10.2019.p9420.